

Professional Coding Specialist

COS Pro 파이썬 2 급

18 강- 20 강. 모의고사 4 차

모의고사 4 차 문제 1-10 번

과정 소개

모의고사(YBM 샘플) 문제를 통해 실전에 대비할 수 있다. (회차당 10 문항, 각 문항별 문제 풀이)

학습 목차

모의고사 4 차 문제 1 번
모의고사 4 차 문제 2 번
모의고사 4 차 문제 3 번
모의고사 4 차 문제 4 번
모의고사 4 차 문제 5 번
모의고사 4 차 문제 6 번
모의고사 4 차 문제 7 번
모의고사 4 차 문제 8 번
모의고사 4 차 문제 9 번
모의고사 4 차 문제 10 번

학습 목표

1. YBM IT(www.ybmit.com) 에서 제공하는 COS Pro 파이썬 2 급 샘플 문제 메뉴의 모의고사를 풀어보며 COS Pro 파이썬 2 급 시험에 대비할 수 있다.

1

모의고사 4 차 문제 1 번

문제 1 의 정답

x 표 있는 번호 오름차순 정렬하기

```
def solution(schedule):
    answer = []
    1 for idx, i in enumerate(schedule):
        2 if i == "X":
            answer.append(idx+1)
    return answer
```

idx	0	1	2	3	4	5	6	7	8	9
i	0	X	X	0	0	0	X	0	X	X

학생 번호 = 리스트 번호(idx) + 1

- ① schedule 리스트의 인덱스 번호와 데이터를 쌍으로 가져오는데 enumerate()함수를 사용한다.
 enumerate()함수는 데이터를 가져올 때 인덱스 번호와 데이터 값을 쌍으로 가져오므로 인덱스 번호가 필요할 경우 유용하다.
- ② 가져온 데이터 값인 i 변수에 들어 있는 값이 "X"이면 answer 리스트에 인덱스 번호 +1 을 추가한다.

2

모의고사 4 차 문제 2 번

문제 2 의 정답

체력시험 합격 인원 구하기

```
def func_a(passed, non_passed):
    3 return ( passed > 1 and non_passed ==0 )
```

```
def func_b(scores):
    2 answer = 0
    if scores[0] < 40:
        answer += 1
    if scores[1] < 44:
        answer += 1
    if scores[2] < 35:
        answer += 1
    return answer
```

```
def func_c(scores):
    1 answer = 0
    if scores[0] >= 80:
        answer += 1
    if scores[1] >= 88:
        answer += 1
    if scores[2] >= 70:
        answer += 1
    return answer
```

```
def solution(scores):
    answer = 0
    for my_score in scores:
        passed = func_c(my_score)
        non_passed = func_b(my_score)
        answer += func_a(passed, non_passed)
    return answer
```

1. 통과한 종목이 몇 개인지 셉니다.
2. 통과 점수의 반을 넘기지 못한 종목이 몇 개인지 셉니다.
3. 통과한 종목이 하나보다 많고 통과 점수의 반을 넘기지 못한 종목이 없으면 통과한 인원으로 셉니다.

① 통과한 종목의 개수를 셉니다.

<func_c 함수>

```
def func_c(scores):
    answer = 0
    if scores[0] >= 80:
        answer += 1
    if scores[1] >= 88:
        answer += 1
    if scores[2] >= 70:
        answer += 1
    return answer
```

윗몸 일으키기 (scores[0]), 팔굽혀 펴기(scores[1]), 달리기(scores[2]) 점수가 각각 합격 점수 이상이면 answer 를 1 증가시킴으로써 통과한 종목의 개수를 센다.

② 통과 점수의 반을 넘기지 못한 종목이 몇 개인지 셉니다.

<func_b 함수>

```
def func_b(scores):  
    answer = 0  
    if scores[0] < 40:  
        answer += 1  
    if scores[1] < 44:  
        answer += 1  
    if scores[2] < 35:  
        answer += 1  
    return answer
```

윗몸 일으키기 (scores[0]), 팔굽혀 펴기(scores[1]), 달리기(scores[2]) 각각의 점수가 합격 점수의 절반 미만이면 answer 를 1 증가시킴으로써 통과 점수의 반을 넘기지 못한 종목의 개수를 센다.

③ 통과한 종목이 하나보다 많고 통과 점수의 반을 넘기지 못한 종목이 없으면 통과한 인원으로 셉니다.

<func_a 함수>

```
def func_a(passed, non_passed):  
    return ( passed > 1 and non_passed ==0 )
```

논리 연산자를 사용하면 true 혹은 false 값을 리턴해 준다. 따라서 통과한 종목 passed 가 1 보다 크고 통과 점수의 반을 넘기지 못한 종목이 없으면 즉 non_passed 가 0 이면 true , 아니면 false 를 리턴한다.

[4 차 2 급 2_solution_code.py]

```
def solution(scores):  
    people_count = 0  
    pass_score = [80, 88, 70]  
  
    for score in scores:  
        pass_count = 0  
        for i in range(3):  
            if score[i] < pass_score[i]/2:  
                pass_count = 0  
                break  
            elif score[i] >= pass_score[i]:  
                pass_count += 1  
        if pass_count >1:  
            people_count += 1  
    return people_count
```

※ 정답 코드에는 문제와 다른 형태의 코드가 들어 있으며, 다른 방식의 코드도 참고해 보세요.

3

모의고사 4 차 문제 3 번

문제 3 의 정답

카드 게임 승자와 점수 구하기

```
def func_a(bundle, start):
    ❶ return bundle[start::2]
```

```
def func_b(score1, score2):
    ❸ if score1 > score2:
        return [1, score1]
    elif score2 > score1:
        return [2, score2]
    else:
        return [0, score1]
```

```
def func_c(bundle):
    ❷ answer = 0
    score_per_cards = {
        'a': 1,
        'b': 2,
        'c': 3,
        'd': 4,
        'e': 5
    }
    for card in bundle:
        answer += score_per_cards[card]
    return answer
```

```
def solution(n, bundle):
    a_cards = func_a(bundle,0) [:n]
    b_cards = func_a(bundle,1) [:n]
    a_score = func_c(a_cards)
    b_score = func_c(b_cards)
    return func_b(a_score,b_score)
```

- ❶ A 와 B 가 번갈아 가며 가져간 카드를 각각 리스트에 넣습니다.
- ❷ A 와 B 가 각각 획득한 점수를 구합니다.
- ❸ 획득한 점수가 큰 사람과 획득한 점수를 순서대로 리스트에 담아 return 합니다.

뽑아야 하는 카드 개수	카드 묶음 (순서대로)	A 승리 : 1 B 승리 : 2 무승부 : 0	승리한 사람의 점수
n	bundle	return	
4	"cacdbdedccbb"	[0, 13]	

① A 와 B 가 번갈아 가며 가져간 카드를 각각 리스트에 넣습니다.

<func_a 함수>

```
def func_a(bundle, start):  
    return bundle[start::2]
```

A 와 B 가 번갈아 가며 카드를 가져가므로 A 는 0 번부터 시작해서 짝수 인덱스 번호에 적혀 있는 카드를 가져가고, B 는 1 번부터 시작해서 홀수 인덱스 번호에 해당하는 카드를 가져가면 된다.

따라서 시작 인덱스 번호에서 시작해서 2 씩 증가한 카드를 가져가도록 코드한다.

solution 함수에서 아래와 같이 호출하는 문장의 표현식에 주의한다.

```
a_cards= func_a(bundle,0)[:n]  
b_cards= func_a(bundle,1)[:n]
```

- func_a(bundle,0) : 0 번부터 시작해서 카드 묶음의 짝수 번호 인덱스의 모든 카드를 가져온다.
- func_a(bundle,0)[:n] : 0 번부터 시작해서 카드 묶음의 짝수 번호 인덱스의 모든 카드 중에 총 n 개의 카드만 골라낸다.

② A 와 B 가 각각 획득한 점수를 구합니다.

<func_c 함수>

```
def func_c(bundle):  
    answer = 0  
    score_per_cards = {  
        'a': 1,  
        'b': 2,  
        'c': 3,  
        'd': 4,  
        'e': 5  
    }  
    for card in bundle:  
        answer += score_per_cards[card]  
    return answer
```

각 카드별 점수가 들어 있는 score_per_cards 변수를 딕셔너리 구조를 이용해 정의한다. 'a'가 적힌 카드는 1 점, 'b'가 적힌 카드는 2 점... 과 같이 각 알파벳에 해당하는 점수를 초기화 한다.

solution 함수에서 아래와 같이 호출하는 문장의 표현식에 주의한다.

```
a_score= func_c(a_cards)
b_score= func_c(b_cards)
```

func_c의 bundle에는 A가 가진 카드인 a_cards 혹은 B가 가진 카드인 b_cards가 넘겨진다.

따라서 받은 카드의 알파벳을 키로 하는 데이터 값을 answer에 합한다.

예를 들어 answer+=score_per_cards['c']가 오면 answer+=3과 같이 score_per_cards 딕셔너리의 'c' 키에 해당하는 값이 더해지게 된다.

③ 획득한 점수가 큰 사람과 획득한 점수를 순서대로 리스트에 담아 return 합니다.

<func_b 함수>

```
def func_b(score1, score2):
    if score1 > score2:
        return [1, score1]
    elif score2 > score1:
        return [2, score2]
    else:
        return [0, score1]
```

```
return func_b(a_score,b_score)
```

획득한 점수가 큰 사람의 번호와 획득한 점수를 순서대로 리스트에 담아 리턴한다.

4

모의고사 4 차 문제 4 번

문제 4 의 정답

수업을 하기 위한 조교의 수 구하기

```
def solution(classes, m):
    answer = 0
    1 for students in classes:
    2     answer += students // m
        if students % m != 0:
            answer += 1
    return answer
```

반별 학생수	조교 1명당 담당하는 학생수	필요한 조교의 수
classes	m	return
[80, 45, 33, 20]	30	8

- ① classes 리스트에서 단어를 하나씩 가져와서 students 변수에 넣는다.
- ② 반별 학생수(students)를 조교 1 명당 담당하는 학생 수(m)로 나눈 몫을 이용해 필요한 조교의 수를 먼저 구한다.
- 반별 학생수(students)를 조교 1 명당 담당하는 학생수(m)로 나눈 값이 0 이 아니면 남은 학생이 존재하므로 조교 1 명이 추가로 필요하다. 따라서 조교의 수(answer)를 1 증가시킨다.

문제 5의 정답

운동으로 소모하는 총 열량 구하기

```
def solution(calorie):
    min_cal = 1000    calorie[0]
    answer = 0
    1 for cal in calorie:
        2 if cal > min_cal:
            answer += cal - min_cal
        3 min_cal = min(min_cal, cal)
    return answer
```

식단의 열량을 담고 있는 리스트 운동으로 소모하는 총 열량

calorie	return
[713, 665, 873, 500, 751]	459

- ① calorie 리스트의 데이터를 하나씩 읽어 와서 cal 변수에 넣는다.
- ② cal(현재 일의 칼로리)이 min_cal(이전까지의 최소값)보다 크면 운동을 한다.
따라서 현재 일의 칼로리에서 이전까지의 최소값을 뺀 값을 운동으로 소모하는 총 열량 변수인 answer에 증가시킨다.
- ③ cal(현재 일의 칼로리)와 min_cal(이전까지의 최소값) 중 더 작은 값을 최소값으로 변경하여 현재까지 값 중 가장 작은 값을 구한다.

이 문제에서는 min_cal(이전까지의 최소값)을 초기화 하는 문제로, 최소값을 구할 때는 초기값을 1000 과 같이 매우 큰 값으로 주거나 첫 번째 데이터를 초기값으로 준다.

6

모의고사 4 차 문제 6 번

문제 6 의 정답

최대 사용 가능한 포인트 구하기

```
def solution(point):  
    ① if point < 1000:  
        return 0  
    ② return point // 100 * 100
```

- ① 포인트가 1000 보다 작으면 0 을 리턴한다.
- ② 포인트가 1000 이상이면 백단위로 포인트를 사용할 수 있다. 따라서 (포인트를 100으로 나눈 몫) * 100 을 하면 된다.

예를 들어 (2323 // 100) 은 23 이고 여기에 100 을 곱하면 2300 이 되므로 2300 포인트를 사용할 수 있다.

7

모의고사 4 차 문제 7 번

문제 7 의 정답

(기말점수-중간점수) 최대, 최소 구하기

```
def func_a(scores1, scores2):
    ① answer = 0
    for score1, score2 in zip(scores1, scores2):
        answer = max(answer, score2 - score1)
    return answer
```

```
def func_b(scores1, scores2):
    ② answer = 0
    for score1, score2 in zip(scores1, scores2):
        answer = min(answer, score2-score1 )
    return answer
```

```
def solution(mid_scores, final_scores):
    up = func_a(mid_scores, final_scores)
    down = func_b(mid_scores, final_scores)
    ③ answer = [up, down]
    return answer
```

- ① 각 학생에 대하여 기말고사 점수에서 중간고사 점수를 뺀 값의 최댓값을 구합니다.
- ② 각 학생에 대하여 기말고사 점수에서 중간고사 점수를 뺀 값의 최솟값을 구합니다.
- ③ 1 번과 2 번 과정에서 구한 점수를 리스트에 담아 return 합니다.

- ① 각 학생에 대하여 기말고사 점수에서 중간고사 점수를 뺀 값의 최댓값을 구합니다.

<func_a 함수>

```
def func_a(scores1, scores2):
    answer = 0
    for score1, score2 in zip(scores1, scores2):
        answer = max(answer, score2 - score1)
    return answer
```

성적이 가장 많이 오른 점수를 구해야 하므로 (기말고사-중간고사) 중 최댓값을 구한다.

중간고사 점수(mid_scores)와 기말고사 점수(final_scores)를 각각 func_a 의 score1 과 score2 매개 변수로 전달한다. 이 두 데이터를 zip 함수를 이용해서 인덱스 번호가 같은 것끼리 쌍으로 가져온다. 인덱스 번호가 같아야 동일한 학생의 데이터 이므로 zip 함수를 사용하면 효과적이다.

- ② 각 학생에 대하여 기말고사 점수에서 중간고사 점수를 뺀 값의 최솟값을 구합니다.

<func_b 함수>

```
def func_b(scores1, scores2):  
    answer = 0  
    for score1, score2 in zip(scores1, scores2):  
        answer = min(answer, score2-score1)  
    return answer
```

성적이 가장 많이 떨어진 점수를 구한다. 이는 ①과 동일한 코드이며, (기말고사-중간고사) 점수 중 최솟값을 구해야 하므로 함수만 max 를 min 으로 수정하면 된다. 여기서는 (중간고사-기말고사)로 코드가 잘못되어 있다. 따라서 (기말고사-중간고사)이므로 (score2-score1)로 고치면 된다.

- ③ 1 번과 2 번 과정에서 구한 점수를 리스트에 담아 return 합니다.

1 과 2 번에서 구한 최댓값과 최솟값을 리턴하면 된다.

[4 차 2 급 7_solution_code.py]

```
def solution(mid_scores, final_scores):  
  
    answer = []  
    answer.append(max(final-mid for mid,final in list(zip(mid_scores,final_scores))))  
    answer.append(min(final-mid for mid,final in list(zip(mid_scores,final_scores))))  
  
    return answer
```

정답지에는 문제와 전혀 다른 방식으로 코드되어 있으므로, 이런 방식의 코드도 참고해 보세요.

8

모의고사 4 차 문제 8 번

문제 8 의 정답

과반수를 득표한 후보자의 번호

```
def solution(n, votes):
    ① arr = [0] * (n + 1)
    ② for vote in votes:
        arr[vote] += 1

    ③ for i in range(1, n+1):
        if arr[i] >= len(votes)/2 and arr[i] == max(arr):
            return i
    return -1
```

후보수	투표 결과	당선자 번호
n	votes	return
3	[1,2,1,3,1,2,1]	1
2	[2,1,2,1,2,2,1]	2

후보자 번호	0	1	2	3
득표수	0	4	2	1

- ① 후보자별 득표수를 저장하기 위한 arr 변수를 후보 수 +1 만큼 0 으로 초기화한다.
- ② 투표 결과(votes) 리스트의 값을 하나씩 가져와서 arr 리스트의 후보 번호 인덱스에 개수를 증가시킨다.
- ③ 후보자별로 얻은 득표수를 가져와서 현재 후보자의 득표수가 최고 득표수이고, 과반수를 넘으면 해당 후보 번호를 리턴한다.

맨 마지막에 return -1 문장은 과반수를 넘지 않으면 -1 을 리턴하기 위한 문장이다.

[4 차 2 급 8_solution_code.py]

```
def solution(n, votes):
    answer = 0
    votes_len = len(votes)
    candidate = votes[0]
    count = 1
    for i in range(1, votes_len) :
        if candidate == votes[i] :
            count += 1
        else :
            count -= 1
            if count == 0 :
                candidate = votes[i]
                count = 1
```

```
test_count = 0
for i in range(0, votes_len) :
    if votes[i] == candidate :
        test_count += 1

if test_count > votes_len // 2 :
    answer = candidate
else :
    answer = -1

return answer
```

정답지에는 문제와 전혀 다른 방식으로 코드되어 있으므로, 이런 방식의 코드도 참고해 보세요.

9

모의고사 4 차 문제 9 번

문제 9 의 정답

위험 지역의 개수 구하기

[2 급 4 차 9 번 다른 코딩 제안-1.py]

```
def solution(height):
    #여기에 코드를 작성해주세요.
    count = 0
    1 di=[-1,1,0,0]
    dj=[0,0,-1,1]
    2 for i in range(4):
        for j in range(4):
            is_danger=True
            for k in range(4):
                if 0<=i+di[k]<4 and 0<=j+dj[k]<4 :
                    if height[i+di[k]][j+dj[k]] <= height[i][j]:
                        is_danger=False
            if is_danger==True:
                count+=1
    return count
```

(0,0) 3	(0,1) 6	(0,2) 2	(0,3) 8	상 : (i-1 , j) 하 : (i+1 , j) 좌 : (i , j-1) 우 : (i , j+1)
(1,0) 7	(1,1) 3	(1,2) 4	(1,3) 2	
(2,0) 8	(2,1) 6	(2,2) 7	(2,3) 3	
(3,0) 5	(3,1) 3	(3,2) 2	(3,3) 9	

"상하좌우 좌표"
di = [-1, 1, 0, 0]
dj = [0, 0, -1, 1]

- ① 상하좌우 좌표 규칙을 초기화한다.
- ② 2 차원 리스트 (i,j)를 돌면서 현재(i,j)를 기준으로 상하좌우의 좌표가 존재하고 모든 값이 기준 값보다 크면 위험 지역으로 카운트한다.

[4 차 2 급 9_solution_code.py]

```
def solution(height):
    count = 0
    dx = [ -1, 1, 0, 0 ]
    dy = [ 0, 0, -1, 1 ]
    for i in range(4):
        for j in range(4):
            is_danger = True
            for k in range(4):
                if 0 <= i+dx[k] and i+dx[k] < 4 and 0 <= j+dy[k] and j+dy[k] < 4:
                    if height[i+dx[k]][j+dy[k]] <= height[i][j]:
                        is_danger = False
            if is_danger:
                count += 1
    return count
```

※ 9 번의 정답지에 있는 코드와 동일하나 x, y 좌표를 이용할 경우 수학에서의 그래프와 혼동될 수 있어 위 정답에는 i, j로 표현하였음.

[2 급 4 차 9 번 다른 코딩 제안-2.py]

```
def solution(height):
    # 여기에 코드를 작성해주세요.
    count = 0
    di=[-1,1,0,0]
    dj=[0,0,-1,1]
    for i in range(4):
        for j in range(4):
            for k in range(4):
                if 0<=i+di[k]<4 and 0<=j+dj[k]<4 :
                    if height[i+di[k]][j+dj[k]] <= height[i][j]:
                        break
            else:
                count+=1
    return count
```

10

모의고사 4 차 문제 10 번

문제 10의 정답

합격자 수 구하기

```
def solution(scores, cutline):  
    answer = 0  
    for s in scores:  
        if s >= cutline:  
            answer += 1  
    return answer
```

점수(scores)를 하나씩 읽어 와서 커트라인 이상이면 개수를 증가시킨다.